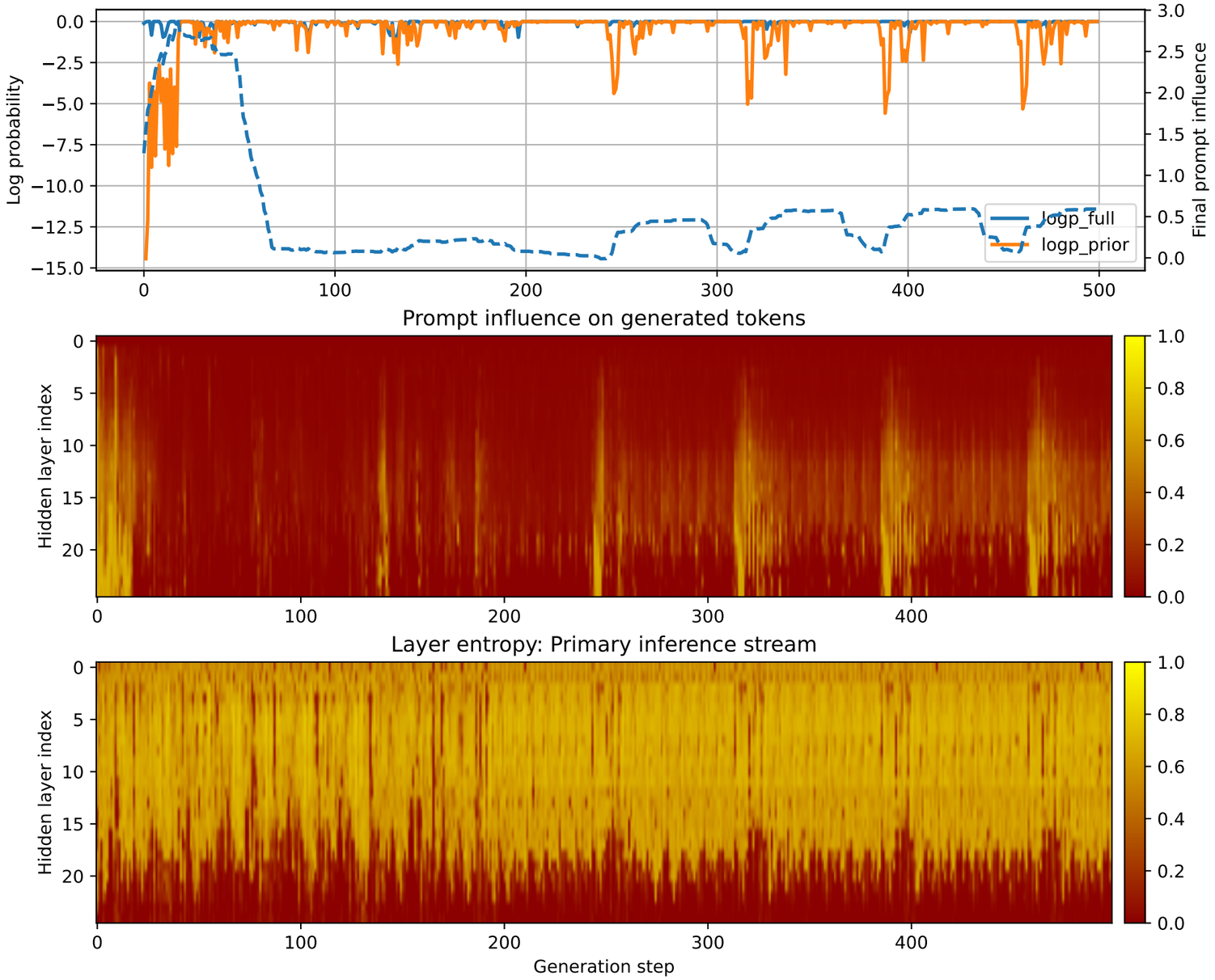
Prompt 1




```
PROMPT:
Task ID: PYC078  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC078  Your code must contain this required symbol:
def is_valid_bst  Implement this task:
define function is_valid_bst(root)..  Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
# CHECK_TOKEN: PyC078  # Define function is_valid_bst(root) def
is_valid_bst(root):     """     Check if a
binary tree is a valid binary search tree (BST).      Args:
  root (Node): The root node of the binary
tree.      Returns:          bool: True if the tree is a valid
BST, False otherwise.     """     # Check if the
root is None     if root is None:          return True      #
Check if the root is a leaf node     if not
root.left and not root.right:          return True      # Check if
the root is a valid BST node     if not
is_valid_bst(root.left) or not is_valid_bst(root.right):
return False      # Recursively check the
left and right subtrees     return is_valid_bst(root.left) and
is_valid_bst(root.right)  # Test the function
root = Node(1) root.left = Node(2) root.right = Node(3)
root.left.left = Node(4) root.left.right = Node(5)
root.right.left = Node(6) root.right.right = Node(7)
print(is_valid_bst(root))  # Output: True  root =
Node(8) root.left = Node(9) root.right = Node(10) root.left.left
= Node(11) root.left.right = Node(12)
root.right.left = Node(13) root.right.right = Node(14)
print(is_valid_bst(root))  # Output: False  root =
Node(15) root.left = Node(16) root.right = Node(17)
root.left.left = Node(18) root.left.right = Node(19)
root.right.left = Node(20) root.right.right = Node(21)
print(is_valid_bst(root))  # Output: False  root =
Node(22) root.left = Node(23) root.right = Node(24)
root.left.left = Node(25) root.left.right = Node(26)
root.right.left = Node(27) root.right.right = Node(28)
print(is_valid_bst(root))  # Output: True  root =
Node(29) root.left = Node(30) root.right = Node(31)
root.left.left = Node(
```